

(Multi-Agent) Reinforcement Learning: a primer, with applications in economics

Games and MARL Basics

Aldo Glielmo

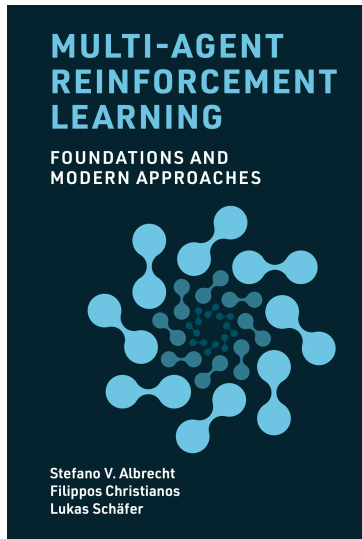
Applied Research Team, Banca d'Italia

A lot of material in this course is based on

Multi-Agent Reinforcement Learning: Foundations and Modern Approaches

The book is freely available from:

www.marl-book.com



Brief Assignment – Thinking about (MA)RL

Task: Send me an email by **March 24th** with:

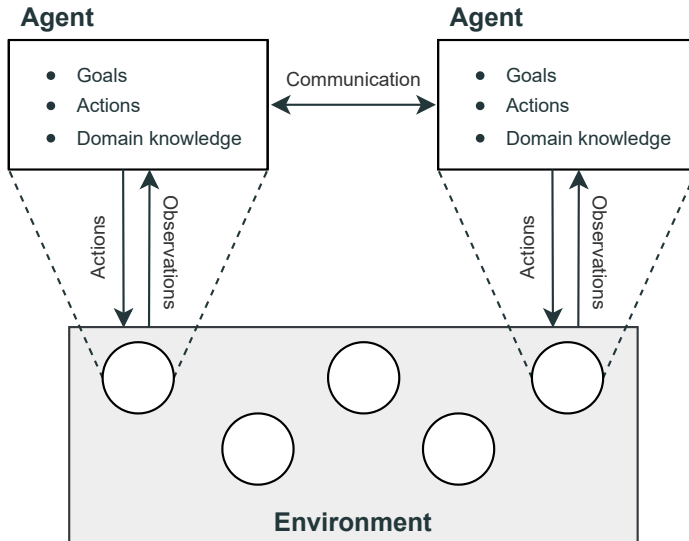
- Your name
- What you liked about the lectures and any open feedback
- Where you're doing your PhD/MSc and with whom
- What your PhD research is about
- How you think RL/MARL could be used in your research (blue sky ideas are fine)

Length: Keep it very short! (100–400 words) It should really not take much.

Why? To learn about your work and explore potential synergies. We can also discuss a few interesting cases in our final lecture on March 25th.

Email: aldo.glielmo@bancaditalia.it

Multi-Agent Systems



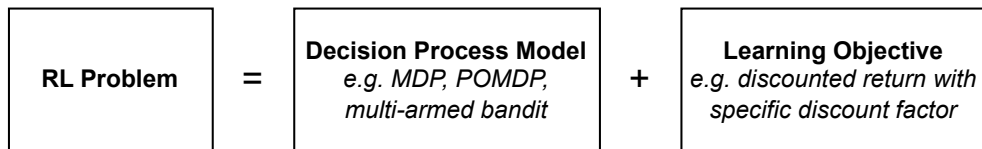
Recap

In the first part, we looked at single agents optimising MDPs via reinforcement learning.

In the following lectures, we will look at multiple agents acting in a shared environment with MARL, with the following topics covered over the next lectures:

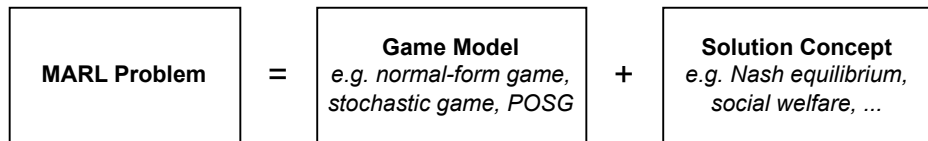
- Today: Multi-agent game models and solution concepts
- Last lecture: Learning and deep MARL methods

Recap: RL Problem Definition



- Sequential decision process is defined formally as **Markov decision process** (MDP)
- Goal is to learn an optimal decision policy for a specific learning objective

MARL Problem Definition



- Sequential decision process is defined formally as a **Game Model**
- Maximizing returns of one agent is no longer enough as a **Solution Concept**
- We must consider the **joint policy** of all agents
- There are different possible solution concepts

Today's Lecture

Game Models

- Normal-form games
 - Stochastic games
 - Partially observable stochastic games
 - Game theory vs MARL assumptions
- ⇒ Short break (10 min, until around 11:25)

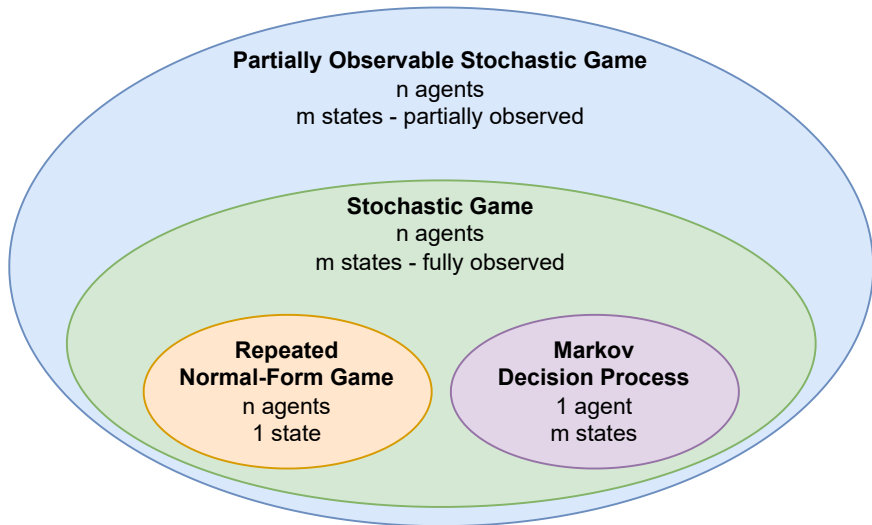
Solution Concepts

- Joint policy and expected return
 - Equilibrium solution concepts
- ⇒ Longer break (15 min, until around 12:30)

Application: Adversarial RL in Economics (Giorgio Bertone & Adriano Meligrana)

Game Models

Hierarchy of Games



Normal-form games define a **single** interaction between two or more agents, providing a simple kernel for more general games to build upon.

Normal-form games are defined as a 3 tuple $(I, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I})$:

- I is a finite set of agents $I = \{1, \dots, n\}$
- For each agent $i \in I$:
 - A_i is a finite set of actions
 - $\mathcal{R}_i$ is the reward function $\mathcal{R}_i : A \rightarrow \mathbb{R}$ where $A = A_1 \times \dots \times A_n$ (set of **joint** actions).

Normal-Form Game Process

In a normal-form game, there are **no time steps or states**. Agents choose an action and observe a reward.

The game proceeds as follows:

1. Each agent samples an action $a_i \in A_i$ with probability $\pi_i(a_i)$
2. The resulting actions from all agents form a **joint action**, $a = (a_1, \dots, a_n)$
3. Each agent receives a reward based on its **individual** reward function and the **joint action**, $r_i = \mathcal{R}_i(a)$

Classes of Games

Games can be classified based on the relationship between the agents' reward functions.

- In **zero-sum games**, the sum of the agents' reward is always 0
i.e. $\sum_{i \in I} \mathcal{R}_i(a) = 0, \forall a \in A$
- In **common-reward** games, all agents receive the same reward ($R_i = R_j; \forall i, j \in I$)
- In **general-sum** games, there are no restrictions on the relationship between reward functions.

Matrix Games

Normal-form games with **2** agents are also called **matrix games** because they can be represented using reward matrices.

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Rock-Paper-Scissors
zero-sum

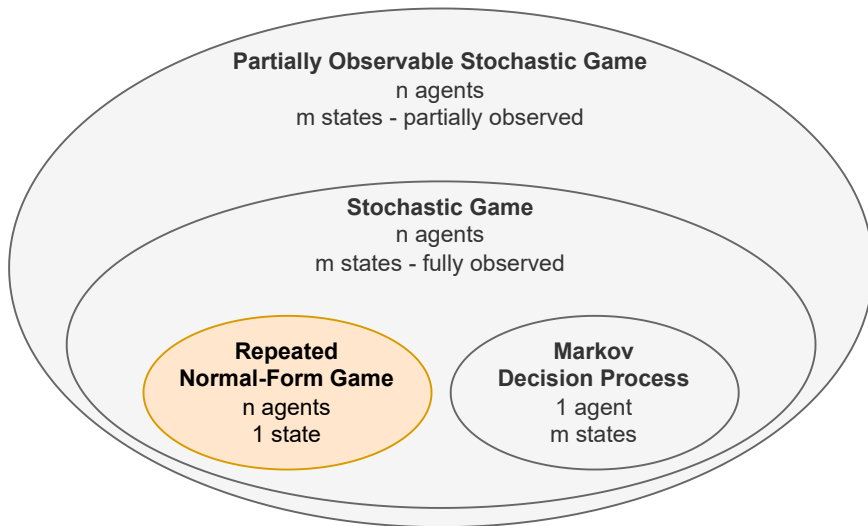
	A	B
A	10	0
B	0	10

Coordination Game
common-reward

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

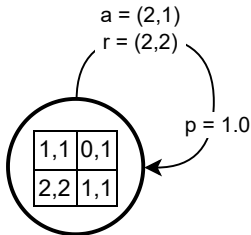
Prisoner's Dilemma
general-sum

Repeated Normal-Form Games



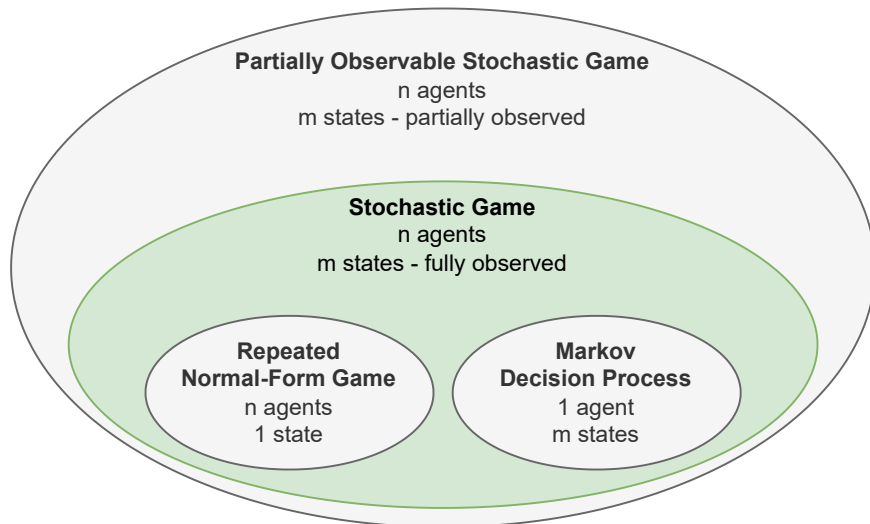
Repeated Normal-Form Games

To extend normal-form games to **sequential** multi-agent interaction, we can repeat the same game over T timesteps.



- At each time step t an agent i samples an action a_i^t
- The policy is now conditioned on a **joint-action** history $\pi_i(a_i^t|h^t)$ where $h^t = (a^0, \dots, a^{t-1})$
- In special cases, h^t contains n last joint actions. E.g. in a tit-for-tat strategy (Axelrod and Hamilton 1981), the policy is conditioned on a^{t-1}

Stochastic Games



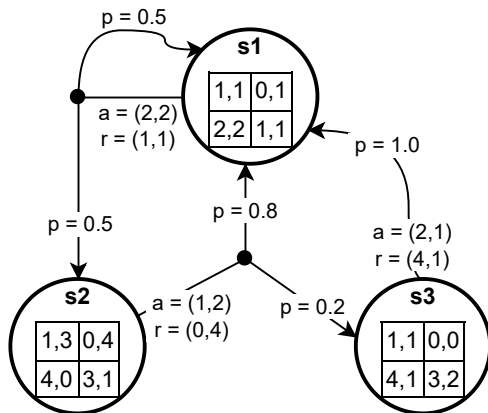
Stochastic Games

Stochastic games introduce the notion of **states** and are defined as a 6 tuple

$$(I, S, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I}, \mathcal{T}, \mu)$$

- I is a finite set of agents
- S is a finite set of states with subset of terminal states $\bar{S} \subset S$
- For each agent $i \in I$:
 - A_i is a finite set of actions
 - $\mathcal{R}_i$ is the reward function $\mathcal{R}_i : S \times A \times S \rightarrow \mathbb{R}$ where A is the set of **joint** actions
 $A = A_1 \times \dots \times A_n$
- $\mathcal{T}$ is the state transition function $\mathcal{T} : S \times A \times S \rightarrow [0, 1]$
- μ is the initial state distribution $\mu : S \rightarrow [0, 1]$

Stochastic Games - Continued

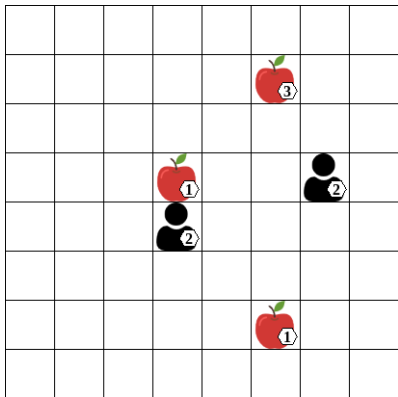


- Each **state** can be viewed as a **non-repeated normal-form game**
- Stochastic games can also be classified into: zero-sum, common-reward or general-sum
- The figure on the left shows a general-sum case

Stochastic Game Process

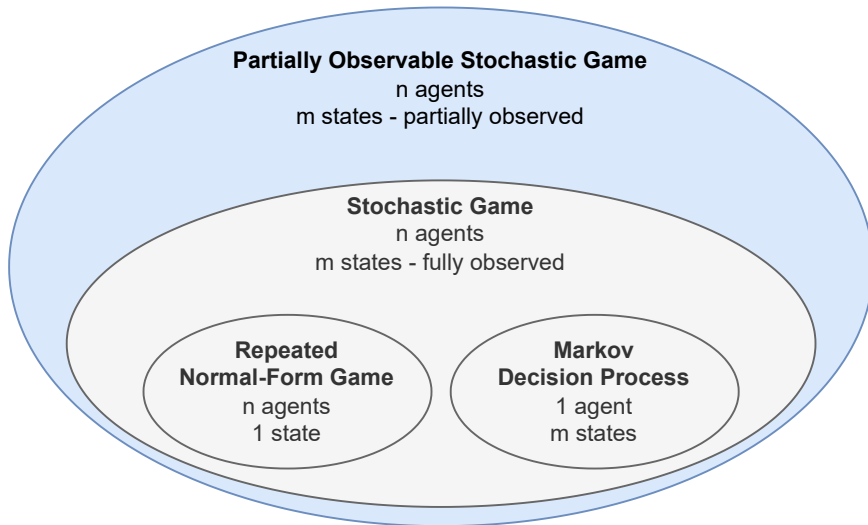
1. Initial state $s_0 \in S$ is samples from μ .
2. At time t each agent $i \in I$ observes the current state $s^t \in S$ and chooses an action $a_i^t \in A_i$ with probability $\pi_i(a_i^t|h^t)$
 - $h^t = (s^0, a^0, s^1, a^1, \dots, s^t)$ is the **state-action history** containing the current state s^t and past **joint** actions and states
3. Given s^t and a^t , the game transitions into the next state $s^{t+1} \in S$ with probability $\mathcal{T}(s^{t+1}|s^t, a^t)$
4. Each agent receives a reward according to their reward function $r_i^t = \mathcal{R}_i(s^t, a^t, s^{t+1})$
5. These steps are repeated until a terminal state $s^t \in \bar{S}$ is reached or a maximum number of T time steps is completed

Example: Level-Based Foraging



- $s \in S$: vector of x-y positions for agents/items, binary collection flags, levels for agents/items
- $a_i \in A_i$: move in four directions, collect item, or no operation (noop)
- $\mathcal{T}$: joint actions update state, e.g., two agents collecting an item switch its flag
- $\mathcal{R}$:
 - common-reward: +1 reward for any item collected by any agent
 - general-sum: +1 reward only for agents directly involved in item collection

Partially Observable Stochastic Games (POSG)



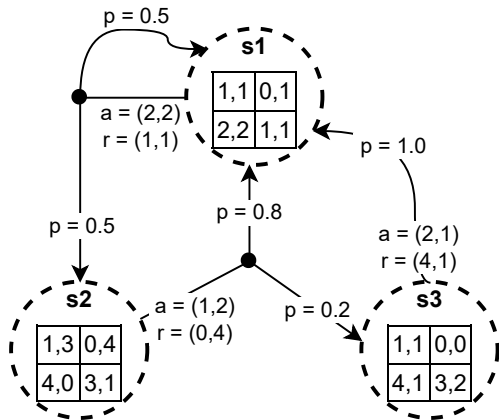
Partially Observable Stochastic Games (POSG)

At the top of the game model hierarchy, the most **general** model is the POSG

- POSGs represent complex decision processes with **incomplete information**
- Unlike in stochastic games, agents receive **observations** providing **incomplete information** about the state and agents' actions
- POSGs apply to scenarios where agents have limited sensing capabilities
 - ⇒ e.g. autonomous driving
 - ⇒ e.g. strategic games (e.g. card games) with private, hidden information

POSG Definition

POSG is defined in the same way stochastic games are, with two additions. Thus it is defined as a 8 tuple $(I, S, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I}, \mathcal{T}, \mu, \{O_i\}_{i \in I}, \{\mathcal{O}_i\}_{i \in I})$



For each agent i we additionally define:

- a finite set of observation O_i
- an observation function $\{\mathcal{O}_i\}_{i \in I}$ such that $\mathcal{O}_i : A \times S \times O_i \rightarrow [0, 1]$ and $\forall a \in A, s \in S : \sum_{o_i \in O_i} \mathcal{O}_i(a, s, o_i) = 1$

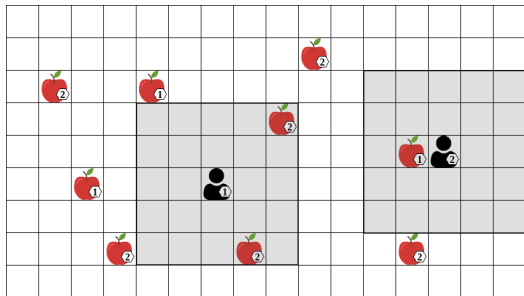
POSG Process

1. Initial state s^0 sampled from $\mu(s)$
2. At each t every agent i receives an **observation** $o_i^t \in O_i$, with probability given by the observation function $\mathcal{O}_i(o^t|s^t, a^{t-1})$
3. Each agent then chooses an action a_i^t based on its policy π_i , which is conditioned on the agent's observation history, $h_i = (o_i^0, \dots, o_i^t)$. All agents' actions give the joint action $a^t = (a_1^t, \dots, a_n^t)$
4. Given the joint action, the environment transitions into the next state s^{t+1} with probability $\mathcal{T}(s^{t+1}|s^t, a^t)$ and each agent receives a reward based on its reward function $r_i^t = \mathcal{R}_i(s^t, a^t, s^{t+1})$
5. This is done until a terminating state $s^t \in \bar{S}$ is reached or a maximum number of time steps is completed

The Observation Function

POSG can represent diverse observability conditions. For example:

- modeling noise by adding uncertainty in the possible observation
- to limit the visibility region of agents (see LBF example)

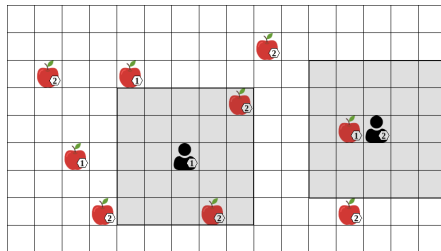


- Here, the agent can only see some parts of the state and joint action
- $o_i^t = (\bar{s}^t, \bar{a}^t)$ where $\bar{s}^t \subset s^t, \bar{a}^t \subset a^t$

Belief States

In partially observable settings, it becomes more challenging to infer optimal actions. For example:

- Optimal action for agent 1 is to move left towards level 1 apple
- But level 1 apple is not directly observable
- Agent 1 can hold a **belief state** b_i^t , a probability distribution over possible state $s \in S$
- Agent 1 might have seen the level-1 apple previously and can thus 'remember' its location



Single Agent Belief Update

To simplify, let's consider the single-agent perspective:

- The initial belief state is given by $b_i^0 = \mu$
- After taking action a_i^t and observing o_i^{t+1} , the belief state b_i^t is updated to b_i^{t+1} using a Bayesian update:

$$b_i^{t+1}(s') \propto \sum_{s \in S} b_i^t(s) \mathcal{T}(s'|s, a_i^t) \mathcal{O}_i(o_i^{t+1}|a_i^t, s')$$

In MARL this type of update is typically **infeasible**:

- High-dimensional state spaces make storage and updates of beliefs intractable
- In MARL for POSG, agents assumed not to know $(S, \mathcal{T}, \mathcal{O}_i)$
- Deep learning can be used to approximate state information (e.g., via a RNN)

Assumptions in Games

Game Theory Assumption

- In game theory, we typically assume that all agents know all components of the game (**complete knowledge games**)
- Agents know all agents' **action spaces and reward functions**
- Knowledge of other agents' reward functions may be used for informing the agent's **best response** action
- Knowledge of the transition function (T) allows for predicting state changes and **planning** actions multiple steps ahead

MARL Assumptions

- In MARL, we assume **limited knowledge**, i.e. no knowledge of transition function $\mathcal{T}$ and no knowledge of agents' reward functions $\mathcal{R}_i$
- Additional assumption can be added and specific knowledge of the game can be held **mutually** or **asymmetrically**
- We usually assume the **number of agents to be fixed**, although recent research has looked at *open* multi-agent systems, this will not be covered in these lectures

Dictionary: Reinforcement Learning $\leftrightarrow$ Game Theory

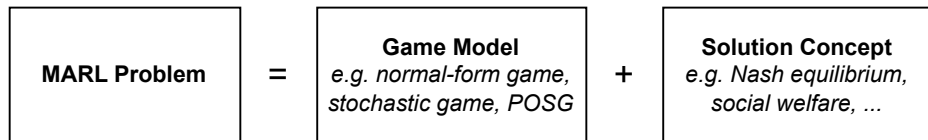
RL		GT
environment	$\leftrightarrow$	game
agent	$\leftrightarrow$	player
reward	$\leftrightarrow$	payoff, utility
policy	$\leftrightarrow$	strategy
deterministic X	$\leftrightarrow$	pure X
probabilistic X	$\leftrightarrow$	mixed X
joint X	$\leftrightarrow$	X profile

- **Environment/Game:** Model with actions, observations, rewards, state dynamics.
- **Agent/Player:** Decision-maker, possibly with specific roles.

- **Reward/Payoff, Utility:** Scalar value received after an action
- **Policy/Strategy:** Assigns probabilities to actions; 'pure strategy' may refer to actions
- **Deterministic X/Pure X:** Assigns probability 1 to X e.g. X = equilibrium or policy
- **Probabilistic X/Mixed X:** Assigns probabilities ≤ 1 to X
- **Joint X/X Profile:** Tuple representing collective aspects, e.g., rewards or policies

Solution Concepts for Games

Solution Concepts for Games



What does it mean to act **optimally** in a multi-agent system?

- Maximizing returns of one agent is no longer enough to determine a solution
- We must consider the **joint policy** of all agents
- This is less straightforward, and there are many different solution concepts

Joint Policy and Expected Return

Joint Policy

The **joint policy** is the combination of all agents' policies:

$$\pi = (\pi_1, \dots, \pi_n)$$

Probability of a joint action under joint policy π is defined as:

$$\pi(a^\tau | h^\tau) = \prod_{j \in I} \pi_j(a_j^\tau | h_j^\tau)$$

Note

This definition assumes probabilistic independence between agents' policies.

Additional Notation

We will use additional notation:

- $\hat{h}^t = \{(s^\tau, o^\tau, a^\tau)_{\tau=0}^{t-1}, s^t, o^t\}$ is the **full history** containing:
 - s^τ , states up to $t - 1$
 - o^τ , joint observations up to $t - 1$
 - a^τ , joint actions up to $t - 1$
 - s^t and o^t at current time step t
- $\sigma(\hat{h}^t) = (o^0, \dots, o^t)$ returns the joint observation history from full history
- $\mathcal{O}(o^t | a^{t-1}, s^t)$ is the joint observation probability

History-Based Expected Return

Given a joint policy π , we can define the expected return for agent i under π as the probability-weighted sum of returns for agent i under all possible **full histories**.

- Let $\hat{H}$ be a set containing all full histories $\hat{h}^t$ for $t \rightarrow \infty$
- Expected return for agent i under joint policy π is given by:

$$U_i(\pi) = \sum_{\hat{h}^t \in \hat{H}} \Pr(\hat{h}^t | \pi) u_i(\hat{h}^t)$$

History-Based Expected Return – Continued

The probability of a full history $\Pr(\hat{h}^t|\pi)$ is:

$$\Pr(\hat{h}^t|\pi) = \mu(s^0)\mathcal{O}(o^0|\emptyset, s^0) \prod_{\tau=0}^{t-1} \pi(a^\tau|\hat{h}^\tau)\mathcal{T}(s^{\tau+1}|s^\tau, a^\tau)\mathcal{O}(o^{\tau+1}|a^\tau, s^{\tau+1})$$

$u_i(\hat{h}^t)$ is the discounted return for agent i in the **full history**

$$u_i(\hat{h}^t) = \sum_{\tau=0}^{t-1} \gamma^\tau R_i(s^\tau, a^\tau, s^{\tau+1})$$

Recursive Expected Return

Expected returns under a joint policy can also be defined *recursively*, analogously to Bellman recursion:

$$V_i^\pi(\hat{h}) = \sum_{a \in A} \pi(a \mid \sigma(\hat{h})) Q_i^\pi(\hat{h}, a)$$

Use this to define a Q -function for agent i as follows:

$$Q_i^\pi(\hat{h}, a) = \sum_{s' \in S} \mathcal{T}(s' \mid s(\hat{h}), a) \left[\mathcal{R}_i(s(\hat{h}), a, s') + \gamma \sum_{o' \in O} \mathcal{O}(o' \mid a, s') V_i^\pi(\langle \hat{h}, a, s', o' \rangle) \right]$$

- $s(\hat{h})$ denotes the last state in $\hat{h}$ such that $s(\hat{h}) = s^t$

Recursive Expected Return – Continued

- $V_i^\pi(\hat{h})$ is the **value (expected return)** for agent i when agents follow **joint policy** π
- $Q_i^\pi(\hat{h}, a)$ is the expected return for agent i when agents execute **joint action** a after $\hat{h}$ and follow π thereafter
- Given the definition for $V_i^\pi(\hat{h})$ and $Q_i^\pi(\hat{h}, a)$, we can define the expected return for agent i at the initial state s^0 as:

$$U_i(\pi) = \mathbb{E}_{s_0 \sim \mu, o_0 \sim \mathcal{O}(\cdot | \emptyset, s_0)} [V_i^\pi(\langle s_0, o_0 \rangle)]$$

Equilibrium Solution Concepts

Best-response policy π_i maximizes expected return for agent i against a given set of policies for all other agents, $\pi_{-i} = (\pi_1, \dots, \pi_{i-1}, \pi_{i+1}, \dots, \pi_n)$

$$\text{BR}_i(\pi_{-i}) = \arg \max_{\pi_i} U_i(\langle \pi_i, \pi_{-i} \rangle)$$

where $\langle \pi_i, \pi_{-i} \rangle$ is the complete **joint policy**

Minimax is a solution concept defined for **two-agent zero-sum** games. Recall that one agent's reward is the negative of the other agent's reward.

- The existence of minimax solution in **normal-form games** was first proven by John von Neumann (1928)
- Minimax solutions also exist in **two-agent zero-sum stochastic games** with finite episode lengths, like chess and Go.

Minimax Definition

In a two-agent zero-sum game, a joint policy $\pi = (\pi_i, \pi_j)$ is a minimax solution if

$$U_i(\pi) = \max_{\pi'_i} \min_{\pi'_j} U_i(\pi'_i, \pi'_j) \quad (1)$$

$$= \min_{\pi'_j} \max_{\pi'_i} U_i(\pi'_i, \pi'_j) \quad (2)$$

$$= -U_j(\pi) \quad (3)$$

- Eq.1 is the maximum expected return agent i can guarantee against any opponent
- Eq.2 is the minimum expected return agent j can **force** on agent i
- A minimax solution is the **best response** to the **worst-case** opponent
- (π_i, π_j) is a minimax solution if $\pi_i \in \text{BR}_i(\pi_j)$ and $\pi_j \in \text{BR}_j(\pi_i)$

Nash Equilibrium

Nash equilibrium solution concept applies the idea of a **mutual best response** to general-sum games with two or more agents.

- No agent i can improve its expected returns by changing its policy π_i assuming other agents policies remain fixed:

$$\forall i, \pi'_i : U_i(\pi'_i, \pi_{-i}) \leq U_i(\pi)$$

- Each agent's policy is a **best response** to all other agent's policies
- John Nash (1950) proved the existence of such a solution for **general-sum non-repeated normal-form games**

Nash Equilibrium in Matrix Games

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

Prisoners Dilemma
NE at D, D (-3, -3)

	A	B
A	10	0
B	0	10

Coordination Game
Two NE's at A, A (10) and
B, B (10)

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Rock Paper Scissors
NE is to choose actions
uniformly at random with
expected return 0

Can you identify the Nash equilibria?

Folk Theorem in Repeated Normal-Form Games

Folk theorems provide solutions for **repeated normal-form games** showing that any **set** of **feasible** and **enforceable** expected returns $\hat{U}$ can be achieved by an equilibrium solution if agents are far-sighted (γ close to 1).

- $\hat{U}$ is **feasible** if it can be achieved by a joint policy π
- $\hat{U}$ is **enforceable** if each $\hat{U}_i$ is at least as great as the agent's minmax value v_i

$$v_i = \min_{\pi_{-i}^m} \max_{\pi_i^m} U_i(\pi_i^m, \pi_{-i}^m)$$

- When $\hat{U}$ is enforceable no agent has an incentive to deviate from π , deviating results in other agents **enforcing** the minmax value $v_i \leq \hat{U}_i$

ϵ -Nash Equilibrium

Exact Nash equilibria are difficult to compute:

- In settings with more than two players, action probabilities may be irrational numbers
- Exact Nash equilibria are often computationally too expensive (PPAD complete problem)
- We can relax the conditions by requiring that no agent can improve its expected return by more than some amount $\epsilon > 0$

Joint policy π is an ϵ -Nash equilibrium for $\epsilon > 0$ if:

$$\forall i, \pi'_i : U_i(\pi'_i, \pi_{-i}) - \epsilon \leq U_i(\pi)$$

ϵ -Nash Equilibrium can be far from Nash Equilibrium

	C	D
A	100,100	0,0
B	1,2	1,1

- Unique Nash equilibrium at (A,C)
- ϵ -Nash equilibrium when $\epsilon = 1$ at (B,D) and (A,C)
- ϵ -Nash equilibrium may not be a good approximation for the true Nash equilibrium
 $\Rightarrow$ Returns under ϵ -Nash equilibrium can differ greatly from returns under the Nash equilibrium

Correlated Equilibrium

Nash equilibrium assumes policies are *independent* $\rightarrow$ this can limit expected returns

Correlated equilibria allow for correlated policies

- π_c is a central policy that assigns probabilities over *joint actions* $a \in A$
- Agents can follow actions 'recommended' by $\pi_c(a)$ or deviate by choosing another action, represented by action modifier $\xi_i : A_i \mapsto A_i$
- Then π_c is a correlated equilibrium if for all i and ξ_i :

$$\sum_{a \in A} \pi_c(a) \mathcal{R}_i(\langle \xi_i(a_i), a_{-i} \rangle) \leq \sum_{a \in A} \pi_c(a) \mathcal{R}_i(a)$$

Correlated Equilibrium Chicken Game

	S	L
S	0,0	7,2
L	2,7	6,6

- Two drivers (agents) drive towards each other on a collision course. Each driver can either swerve (S) or go straight (L). If both swerve, they get a payoff of 6 (both demonstrate courage). If one swerves and the other goes straight, they get payoffs of 2 and 7 respectively (one demonstrates more courage). If both go straight, they crash and get a payoff of 0 (they die).
- A classic movie scene, e.g. *Footloose* (1984) [link to video](#), *Fast and Furious 7* (2015).

Correlated Equilibrium Chicken Game

	S	L
S	0,0	7,2
L	2,7	6,6

Nash Equilibrium (not correlated): Correlated Nash Equilibrium:

- Deterministic: $\pi_i(S) = 1, \pi_j(S) = 0 \rightarrow (7, 2)$ and $\pi_i(S) = 0, \pi_j(S) = 1 \rightarrow (2, 7)$
- Probabilistic: $\pi_i(S) = \frac{1}{3}, \pi_j(S) = \frac{1}{3} \rightarrow \approx (4.66, 4.66)$
- $\pi_c(L, L) = \pi_c(S, L) = \pi_c(L, S) = \frac{1}{3}$ and $\pi_c(S, S) = 0$
- Expected return $= 7 \cdot \frac{1}{3} + 2 \cdot \frac{1}{3} + 6 \cdot \frac{1}{3} = 5$
- Assuming knowledge of π_c , if agent i receives recommendation L they know agent j will choose either S or L with probability 0.5
- Expected return is $0.5 \cdot 6 + 0.5 \cdot 2 = 4$, which is greater than deviating from the action where the agent i has an expected return $0.5 \cdot 0 + 0.5 \cdot 7 = 3.5$

Matrix Games – Interactive Visualisation

- Explorer of matrix games: [link](#)
- Some things to check:
 - ⇒ Can you find/understand the best-response actions?
 - ⇒ Can you find/understand the Nash equilibria?
 - ⇒ Can you guess/understand whether a game is zero-sum, common-reward, or general-sum?
 - ⇒ Can you guess/understand how properties change when you change the payoffs?
 - ⇒ You can also try out some other games ([link](#))

Shortcomings of Equilibrium Solutions

Equilibrium solutions are standard solution concepts in MARL, but have **limitations**.

- **Sub-optimality:**

- Nash equilibria do not always maximize expected returns
- E.g. in Prisoner's Dilemma, (D,D) is Nash but (C,C) yields higher returns

- **Non-uniqueness:**

- There can be multiple (even infinitely many) equilibria, each with different expected returns

- **Incompleteness:**

- Equilibria for sequential games don't specify actions for **off-equilibrium paths**, i.e. paths not specified by equilibrium policy $\Pr(\hat{h}|\pi) = 0$
- If there is a temporary disturbance that leads to an off-equilibrium path, the equilibrium policy π does not specify actions to return to an **on-equilibrium** path

Refinement Concepts

Pareto Optimality

To address some of these limitations, we can add additional solution requirements such as **Pareto optimality**.

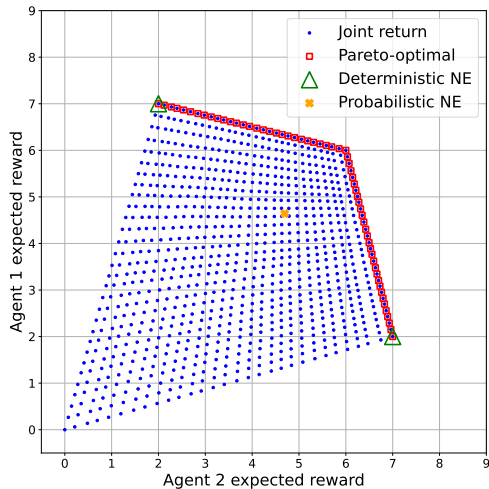
A joint policy π is **Pareto-optimal** if it is not **Pareto-dominated** by any other joint policy. A joint policy π is Pareto-dominated by another policy π' if

$$\forall i : U_i(\pi') \geq U_i(\pi) \quad \text{and} \quad \exists i : U_i(\pi') > U_i(\pi).$$

Intuition

A joint policy is **Pareto-optimal** if there is no other joint policy that improves the expected return for at least one agent without reducing the expected return for any other agent.

Pareto-Optimal Solution in the Chicken Game



	S	L
S	0,0	7,2
L	2,7	6,6

- Figure shows the discretized space of joint policies for Chicken matrix game
- Each blue dot represents the expected joint return obtained by a joint policy

Social Welfare and Fairness

To further constrain the space of desirable solutions, we can consider social welfare and fairness concepts.

Welfare optimality:

$$W(\pi) = \sum_{i \in I} U_i(\pi)$$

- A joint policy π is **welfare-optimal** if $\pi \in \arg \max_{\pi'} W(\pi')$

Fairness optimality:

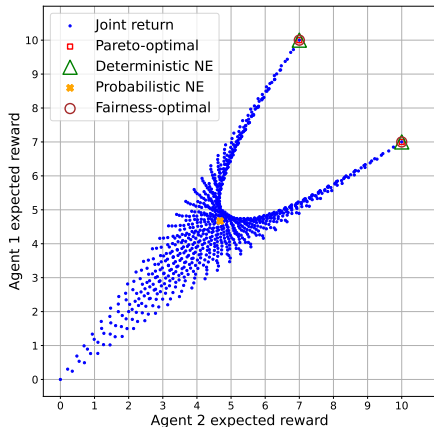
$$F(\pi) = \prod_{i \in I} U_i(\pi)$$

- A joint policy π is **fairness-optimal** if $\pi \in \arg \max_{\pi'} F(\pi')$

Solution and Refinement Concepts – Interactive Visualisation

- Explorer of refinement concepts: [link](#)
- Some things to check:
 - ⇒ Can you find/understand the Nash equilibria?
 - ⇒ Can you find/understand the Pareto-optimal solutions?
 - ⇒ Can you find/understand the welfare-optimal and fairness-optimal solutions?
 - ⇒ Can you guess/understand how properties change when you change the payoffs?
 - ⇒ You can also try out some other games ([link](#))

Fairness in Battle of the Sexes



	A	B
A	10,7	2,2
B	0,0	7,10

- 2 agents agreeing to meet at either location A or B, with each agent having a preference for one or the other location
- A, A and B, B are both Nash equilibria and fairness optimal
- In the chicken game, there is only 1 Pareto optimal and fairness optimal solution

Interactive Coding and Learning Session

References

- *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. See chapters 3 and 4. A possible future “bible” for multi-agent RL. [link](#)
- *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. See chapters 3–6. A classic book for MAS. [link](#)

Computational Resources

- [marl-book code](#)

Game-theoretic Foundations of MAS – Foundational Papers

Original Papers

- Minimax: *Theory Of Games And Economic Behavior*, von Neumann & Morgenstern, *Princeton University Press*, 1944. [link](#)
- Nash eq: *Equilibrium Points in N-Person Games*, Nash, *PNAS*, 1950. [link](#)
- SGs: *Stochastic Games* Shapley, *PNAS*, 1953. [link](#)
- MGs: *Markov games as a framework for multi-agent reinforcement learning*, Littman, *ICML*, 1994. [link](#)
- Correlated eq: *Subjectivity and Correlation in Randomized Strategies*, Aumann, *Journal of Mathematical Economics*, 1974. [link](#)
- Tit-for-Tat / Folk Theorem: *The Evolution of Cooperation*, Axelrod & Hamilton, *Science*, 1981. [link](#)

Computational Resources (Python)

- [Gambit](#) (general game solver)
- [Gymnasium](#) (single-agent interface)
- [PettingZoo](#) (multi-agent interface)

Computational Resources (Julia)

- [GameTheory.jl](#) (general game solver)
- [POMDPs.jl](#) (single-agent interface, but flexible)

References

- *Robust Policy Design in Agent-Based Simulators using Adversarial Reinforcement Learning*, MARW@AAAI 2025. [link](#)
- *Reinforcement Learning in Agent-Based Models: Towards Robust Policy Design*, Giorgio Bertone, Master Thesis @ Sapienza 2025. [link](#)

Computational Resources

- [Python code](#)

Brief Assignment – Thinking about (MA)RL

Task: Send me an email by **March 24th** with:

- Your name
- What you liked about the lectures and any open feedback
- Where you're doing your PhD/MSc and with whom
- What your PhD research is about
- How you think RL/MARL could be used in your research (blue sky ideas are fine)

Length: Keep it very short! (150–400 words) It should really not take much.

Why? To learn about your work and explore potential synergies. To get you thinking about the material we covered. We can also discuss a few interesting cases in our final lecture on March 25th.

Email: aldo.glielmo@bancaditalia.it

Brief Assignment – Possible Structure

Name: [Your name]

PhD/MSc Program: [University] with [Supervisor]

Research Topic: [Brief description]

RL/MARL in My Research: [How could it apply?]

What I Liked and What Could Be Improved: [Any feedback here]

Additional Feedback: [Other thoughts/ideas]

But feel free to structure it as you prefer

Email: aldo.glielmo@bancaditalia.it